EduGenie:Google Gemini Powered Learning Assistant

A project submitted to Manonmaniam Sundaranar University at Tirunelveli in partial fulfilment of the requirements for the degree of

BACHELOR OF SCIENCE

IN

COMPUTER SCIENCE

SUBMITTED BY

NAME : SURAJH KUMAR R B
REG. NO : 25083111802111153

TN SKILL PROJECT

DEPARTMENT OF COMPUTER SCIENCE

NESAMONY MEMORIAL CHRISTIAN COLLEGE MARTHANDAM,

KANYAKUMARI-629 165

2026-2027

EduGenie: Google Gemini Powered Learning Assistant

NESAMONY MEMORIAL CHRISTIAN COLLEGE MARTHANDAM, KANYAKUMARI- 629165

MANONMANIAM SUNDARANAR UNIVERSITY, TIRUNELVELI



Submitted by

Team ID : SWTID-2026-8199

Team Leader :	Surajh Kumar R B	(25083111802111153)
Team member :	Shelton Roy T V	(25083111802111148)
Team member :	Vineesh V	(25083111802111154)

Under the guidance of

Mrs.Prema Bai M.C.A.,B.Ed.,M.Phil.,NET.

Department of Computer Science (Aided)

Student Name	Surajh Kumar R B
Department	B.Sc. Computer Science (Aided)
Naan Mudhalvan ID	36B42208FC1E02083894C28EA084A8D6
Register number	25083111802111153
College Name & College Code	Nesamony Memorial Christian College, Marthandam & 311
Project Title	*EduGenie: Google Gemini Powered Learning Assistant*

Signature of the HOD

Internal Signature **External signature**

ACKNOWLEDGEMENT

I wish to express my sincere gratitude to the Principal Dr.C.H.Arun M.Sc., M.Phil., Ph.D, Nesamony Memorial Christian College, Marthandam.

I sincerely acknowledge my heartfelt thanks to our SPOC Dr. R.Jeba Raj M.Sc., M.Phil., SET., for his support throughout the course of the TN SKILL Program.

I wish to extend my sincere thanks to Head of the Department Er.Felix Anderson Christdhas M.E of Computer Science Aided stream for his continuous encouragement and guidance.

I also wish to extend my sincere thanks to Mrs.H.Prema Bai M.C.A., B.Ed., M.Phil., NET., Department of Computer Science for her constant support, valuable suggestion and encouragement throughout the course.

TABLE OF CONTENTS

1. Pre-requisites ..... **Page 6**

1.1 Knowledge Pre-requisites 6
1.2 Software Requirements 6
1.3 Accounts, Keys and Network Access 7
1.4 Hardware Requirements 7
1.5 Environment Preparation (Step by Step) 8

2. Project Workflow ..... **Page 9**

2.1 End-to-End Request Workflow 9
2.2 Task Routing 9
2.3 Start-up Workflow 10
2.4 Development Workflow 10
2.5 Why this Workflow is Useful 10

3. Model Selection and Architecture ..... **Page 11**

3.1 Model Selection and Justification 11
3.2 System Architecture 11
3.3 Module Responsibilities 12
3.4 Key Architectural Decisions 12

4. Core Functionalities Development ..... **Page 14**

4.1 Feature Overview 14
4.2 Concept Explanation - Local Model 14
4.3 Gemini-Based Features 15
4.4 Backend Routes and Routing Logic 15
4.5 Validation and Error Handling 15

5. Frontend Development ..... **Page 17**

5.1 Technologies Used 17
5.2 Page Structure 17
5.3 How the Template Connects to the Backend 17
5.4 Styling (style.css) 18
5.5 Current Limitations and Possible Improvements 19

6. Testing and Deployment ..... **Page 20**

6.1 Introduction 20
6.2 Testing 20
6.3 Deployment 22

7. Conclusion ..... **Page 24**

7.1 Summary of the Project 24
7.2 Objectives Achieved 24
7.3 Limitations 25
7.4 Future Enhancements 25

Pre-requisites

EduGenie - AI Learning Assistant

1. Purpose of this Document

EduGenie is a multi-model AI learning portal. It combines a FastAPI web server, a small HTML/CSS front end, a local Hugging Face model and the Gemini cloud API. Because several technologies work together, a developer needs the right knowledge, software, accounts and hardware before starting. This document lists each requirement, explains *why* it is needed in this project, and shows how to prepare the environment step by step.

2. Knowledge Pre-requisites

The table links each skill to the exact place where it is used in the EduGenie code, so a new team member knows what to study first.

Skill	Where it is used in EduGenie	Why it matters
Python 3	All modules (Main.py, qna.py, quiz_module.py ...)	Functions, type hints, Enum, exceptions and environment variables are used throughout.
FastAPI	Main.py: app, @app.get, @app.post, Form(...)	Defines the two routes and receives the form data sent by the browser.
Jinja2 templating	templates/index.html: {{ response }}, {% if response %}	Lets the server insert the answer, the prompt and the model name into the page.
HTML and CSS	index.html and static/style.css	Builds the form, the answer box and the visual styling.
Prompt writing	Every module builds a formatted_prompt	The quality and layout of each answer depend on clear instructions to the model.
Hugging Face Transformers	explanation_module.py	Loads a tokenizer and a T5 model and generates text on the CPU.
REST/API basics	google-genai client in Main.py	The Gemini features call a cloud API with a secret key.
Command line and Git	Running uvicorn, pip, version control	Needed to install, run and share the project.

3. Software Requirements

All Python packages are listed in requirements.txt and are installed with one command. The 'Role in EduGenie' column explains what each package actually does in the application.

Package	Version	Role in EduGenie
Python	3.9 or higher	Runs every module of the application.
fastapi	>= 0.110.0	Web framework: routing, form handling, template responses.

Package	Version	Role in EduGenie
uvicorn	>= 0.28.0	ASGI server that listens for browser requests and runs the app.
jinja2	>= 3.1.3	Renders index.html with the values passed from Main.py.
python-multipart	>= 0.0.9	Required by FastAPI to read HTML form fields (prompt, task).
python-dotenv	>= 1.0.1	Reads GEMINI_API_KEY from the .env file so the key is not hard-coded.
google-genai	>= 0.1.1	Official client used to call the Gemini model.
transformers	>= 4.38.0	Downloads and runs the LaMini-Flan-T5-783M model.
torch	>= 2.2.0	Deep-learning engine that executes the local model.
sentencepiece	>= 0.2.0	Tokenizer library needed by T5-family models.
protobuf	>= 4.25.0	Supporting library required by the tokenizer and model loading.

4. Accounts, Keys and Network Access

- **Gemini API key (Google AI Studio):** Main.py reads GEMINI_API_KEY (or GOOGLE_API_KEY). If neither exists the program raises a RuntimeError and refuses to start, so the problem is found immediately rather than when a student submits a question.
- **Key type check:** a value beginning with "ya29." is an OAuth access token, which the Gemini client does not accept. The code detects this case and explains that an AI Studio API key is needed.
- **Internet for the first run:** the local model (about 783 million parameters) is downloaded from Hugging Face the first time the Concept task is used, and then reused from the local cache.
- **Internet during use:** the four Gemini tasks (Q&A, summary, quiz, learning path) need a connection on every request.

5. Hardware Requirements

Component	Minimum	Recommended	Reason
CPU	Dual-core	Quad-core or better	The local model runs on CPU (no GPU code in the project).
RAM	4 GB	8 GB or more	The model and PyTorch are held in memory after the first concept request.
Disk	4 GB free	6 GB or more	PyTorch, Transformers and the downloaded model files.
GPU	Not required	Optional	The code does not use one, so it is a possible future speed-up.

These hardware figures are planning estimates, not measurements. The real download and memory size depend on the PyTorch build installed on your machine.

6. Environment Preparation (Step by Step)

```
1. Check Python:               python --version
2. Create virtual environment: python -m venv venv
3. Activate it:                venv\Scripts\activate     (Windows)
                               source venv/bin/activate    (Linux / macOS)
4. Install packages:           pip install -r requirements.txt
5. Create the .env file:       GEMINI_API_KEY=your_api_key_here
6. Check folder layout:        Main.py, modules, templates/index.html, static/style.css
7. Start the server:           uvicorn Main:app --reload
8. Open the application:       http://127.0.0.1:8000
```

Why a virtual environment? PyTorch and Transformers are large and version-sensitive. A virtual environment keeps them separate from other projects and makes the setup repeatable on another computer.

Why a .env file? It keeps the secret API key outside the source code. Main.py loads it from the same folder as the script using Path(__file__).resolve().parent, so it is found no matter where the command is started. The file must never be uploaded to a public repository.

Project Workflow

EduGenie - AI Learning Assistant

1. Introduction

This document explains how EduGenie works from the moment a student opens the page until the answer is shown, how the work was organised into development stages, and why the workflow is designed this way: **one simple page, many specialised AI tools behind it**.

2. End-to-End Request Workflow

1. Open the portal	The browser sends GET /. Main.py returns index.html with response=None, an empty prompt and task="concept", so the page opens clean with Concept Explanation selected.
2. Choose a task	The student picks one of five options in the drop-down. Each option shows which model will answer, for example "Concept Explanation (Local LaMini-Flan-T5)".
3. Enter the query	The student types a question, topic or text into the textarea (marked required, so the browser blocks an empty submission).
4. Submit	The form posts prompt and task to /generate.
5. Validate	FastAPI converts task into the TaskType enum. Any value outside concept, qa, summary, quiz or path is rejected automatically.
6. Route	An if/elif chain in generate_response() calls the matching module and sets the model label.
7. Generate	Concept: local LaMini model. Other tasks: a task-specific prompt is built and sent to Gemini through generate_with_retry().
8. Handle errors	Any exception is caught; the page shows "Error generating response: ..." with the label "System Error".
9. Display	index.html is rendered again with the answer, the original prompt, the chosen task and a badge showing the model used.

3. Task Routing

Task value	Function called	Model	Typical output
concept	explanation_module.get_concept_explanation	LaMini-Flan-T5-783M (local CPU)	Short explanation (up to 256 new tokens)
qa	qna.get_answer	Gemini (Google Cloud)	Detailed answer with practical examples
summary	summary_module.summarize_text	Gemini (Google Cloud)	1-2 sentence overview and bullet takeaways

Task value	Function called	Model	Typical output
quiz	quiz_module.generate_quiz	Gemini (Google Cloud)	3 multiple-choice questions and an answer key
path	learning_path.generate_roadmap	Gemini (Google Cloud)	Phased roadmap with topics, projects, timelines

4. Start-up Workflow

- **Load configuration:** python-dotenv reads the .env file next to Main.py.
- **Check the key:** the API key must exist and must not be an OAuth token; otherwise start-up stops with a clear message.
- **Create one Gemini client:** a single client is created once and passed into every Gemini-based function.
- **Mount resources:** /static is served for style.css and templates/ is registered for Jinja2.
- **Keep the local model idle:** the LaMini model is not loaded yet. It loads on the first Concept request, so the server starts quickly and uses no model memory unless that feature is used.

5. Development Workflow

The project was built in three stages, called epics. Each one produces something the next one depends on, so the order matters.

Stage	Question answered	Result	Used by
Model Selection and Architecture	Which models, and how should the code be organised?	Hybrid local + Gemini design, one module per task, layered structure	Core Functionalities
Core Functionalities Development	How does each learning tool actually work?	Five task modules, prompts, validation, routes, error handling	Frontend
Frontend Development	How does the student use all this?	index.html form, style.css, answer box with model badge	Testing and Deployment

6. Why this Workflow is Useful

- **Clear separation:** the page knows nothing about models; the modules know nothing about HTML. Either side can be improved on its own.
- **Predictable:** every task follows the same path, so errors are easy to trace.
- **Cost control:** the free local model handles the simplest task, and the paid cloud model is used only where it adds value.
- **Easy extension:** a new tool needs one module, one enum value, one routing branch and one drop-down option.

Model Selection and Architecture

EduGenie - AI Learning Assistant

1. Objective

This stage decided which AI models would power EduGenie and how the application would be structured around them. The aim was a design that is cheap to run, easy to understand, robust against failures and simple to extend with new learning tools.

2. Design Requirements

- Offer five different study aids (explain, answer, summarise, quiz, roadmap) from one interface.
- Keep at least one feature working without paying per request.
- Return readable, well-structured answers suited to students.
- Let the user see which model produced each answer.
- Fail gracefully when a model, the network or the API key has a problem.

3. Model Selection and Justification

Aspect	LaMini-Flan-T5-783M (local)	Gemini Flash (cloud)
Provider / access	MBZUAI, downloaded from Hugging Face	Google, called through the google-genai client
Runs on	The user's own CPU	Google Cloud servers
Cost	No per-request cost	Based on API usage and quota
Strength	Small, fast to start, works offline after download	Follows long, structured instructions; fluent long-form output
Weakness	Short, simpler answers	Needs internet and an API key
Used for	Concept Explanation	Q&A, Summarisation, Quiz, Learning Path

How the choice was made. Concept explanation needs only a few clear sentences, which an instruction-tuned T5 model of this size can produce locally. The other four tasks require strict output layouts - for example a quiz with options A-D and an Answer Key, or a roadmap with phases and timelines - and longer answers. A larger cloud model follows such formatting instructions far more reliably, so those tasks use Gemini.

Why it is useful. This hybrid approach gives students a free tool for quick definitions and a stronger tool for deeper study material, while the interface makes the difference visible through the model badge.

4. System Architecture

Presentation layer	templates/index.html and static/style.css. A form for input and an answer box for output.

API layer	Main.py with FastAPI: GET / and POST /generate, the TaskType enum, start-up checks and error handling.
Feature layer	explanation_module, qna, summary_module, quiz_module, learning_path - one file per learning tool.
Model layer	Local: Transformers tokenizer and AutoModelForSeq2SeqLM. Cloud: one shared Gemini client and the generate_with_retry helper.
Configuration	python-dotenv loads the key; Main.py validates it before the server accepts requests.

5. Module Responsibilities

File	Responsibility
Main.py	Creates the app, mounts static files and templates, validates the API key, defines the TaskType enum, routes requests and renders results.
explanation_module.py	Loads the local model once (lazy loading with module-level cache) and generates concept explanations.
qna.py	Wraps the user's question in a prompt asking for clear, detailed answers with examples.
summary_module.py	Builds a prompt that requests an overview first, then bullet-point takeaways in simple language.
quiz_module.py	Builds a prompt for a 3-question multiple-choice quiz with an Answer Key.
learning_path.py	Builds a prompt for a phased roadmap with topics, project ideas and timelines in Markdown.
gemini_helper.py	Provides generate_with_retry, shared by all Gemini modules (file not included in the code supplied for this document, so only its use is described).

6. Key Architectural Decisions

Decision	Reason	Benefit
One module per task	Each tool has its own prompt and rules	A change to the quiz cannot break the summary
Gemini client created once and passed in as a parameter	Avoids repeated set-up and global state	Modules are easy to test with a fake client
Lazy loading of the local model	Loading takes time and memory	Fast start-up; model loaded only when needed
Module-level cache (_tokenizer, _model)	Avoid reloading on each request	Only the first concept request is slow
Shared retry helper	Cloud calls can fail temporarily	One place handles retries for four features
Fail-fast key validation	A bad key would otherwise fail mid-request	Clear message at start-up

Decision	Reason	Benefit
Model label sent with every answer	Transparency for the learner	The page shows which model answered

7. Outcome

The result is a layered, modular architecture in which a thin FastAPI layer routes each request to a dedicated feature module, and each module talks to the model best suited to the job. This structure is what the next stage, Core Functionalities Development, builds on.

Core Functionalities Development

EduGenie - AI Learning Assistant

1. Objective

This stage turned the architecture into working features. It covers the five learning tools, the routes that connect them to the web page, and the validation and error handling that keep the system stable. For each feature this document explains how it is made and how it helps the learner.

2. Feature Overview

Feature	File	How it is made	How it helps the learner
Concept Explanation	explanation_module.py	Local LaMini-Flan-T5 model via Transformers	Quick plain-language definition without internet cost
Question Answering	qna.py	Gemini with an instruction for detailed, example-rich answers	Deeper understanding of any question
Summarisation	summary_module.py	Gemini with overview + bullet format	Saves reading time on long text
Quiz Generation	quiz_module.py	Gemini with a 3-question MCQ format and Answer Key	Instant self-assessment
Learning Path	learning_path.py	Gemini with phased roadmap format	A study plan with stages, projects and timelines

3. Concept Explanation - Local Model

3.1 How it is made

The function load_model() stores the tokenizer and model in two module-level variables, _tokenizer and _model. If they are empty, it loads MBZUAI/LaMini-Flan-T5-783M with AutoTokenizer and AutoModelForSeq2SeqLM and prints progress messages to the console. On later calls the stored objects are returned, so the model is loaded only once.

get_concept_explanation() then (1) rejects empty input, (2) wraps the text in the instruction "Explain this concept clearly and concisely", (3) tokenises it as PyTorch tensors with truncation at 512 tokens, (4) calls model.generate() and (5) decodes the result without special tokens.

Setting	Value	Purpose
max_length (input)	512	Keeps very long inputs within the model's limit
max_new_tokens	256	Limits answer length and response time
do_sample	False	Greedy decoding gives the same answer for the same input
skip_special_tokens	True	Removes internal markers from the text

3.2 How it is useful

Students get a short explanation immediately, with no API key usage or per-request cost. Because decoding is deterministic, the result is repeatable, which helps when the feature is tested.

4. Gemini-Based Features

The four cloud features share one pattern, which keeps the code small and consistent:

- **Check:** return a friendly message if the input is empty, without calling the API.
- **Build:** create formatted_prompt = task instructions + formatting rules + the user's text (stripped of extra spaces).
- **Call:** send the prompt with generate_with_retry(client, formatted_prompt).
- **Protect:** wrap any failure in a RuntimeError that names the task, such as "Quiz generation failed: ...".

Module	Formatting rules placed in the prompt	Why these rules
qna	Clear answer, detailed explanations, practical examples	Learners understand better with worked examples
summary_module	1-2 sentence overview first, then bullet takeaways, simple language	Easy to scan; the overview gives instant context
quiz_module	3 questions, options A-D, final 'Answer Key' with brief explanations	Gives a fixed, easy-to-read quiz and teaches through explanations
learning_path	Phases (Beginner, Intermediate, Advanced), key topics, project ideas, timelines, Markdown headers and bullets	A roadmap needs order, practice and time estimates

5. Backend Routes and Routing Logic

Route	Method	What it does
/	GET	Renders index.html with response=None, prompt empty and task=concept (the first-visit state).
/generate	POST	Reads prompt and task from the form, calls the matching module, stores the answer and the model label, and renders index.html.

The TaskType enum is the contract between the page and the server. Because task is declared as TaskType in the route signature, FastAPI validates it automatically and the if/elif chain only ever sees one of five legal values. The Gemini model label is stored once in the GEMINI_LABEL constant, so it can be changed in a single place.

6. Validation and Error Handling

Layer	What is checked	What the user sees
Browser	Textarea has the required attribute	The form cannot be sent empty

Layer	What is checked	What the user sees
Start-up	API key present and not an OAuth token	Server refuses to start with a clear message
Module	Input empty after trimming	"Please provide ..." message, no model call
Gemini call	Any exception from the API	RuntimeError naming the failed task
Local model	Any exception while loading or generating	"Error running local model: ..."
Route	Any exception reaching generate_response()	"Error generating response: ..." with the label System Error

A comment in Main.py records an important fix: the final return statement sits outside the try/except block, so the page is returned for both success and error cases. This means the user always gets a rendered page back, with either the answer or an error message.

7. Example Flow: Learning Path

```
Input   : task = path, prompt = "Data Science"
Route   : TaskType.PATH -> learning_path.generate_roadmap(gemini_client, prompt)
Prompt  : "Create a comprehensive, step-by-step learning roadmap ... Phase 1 ... Phase 3 ..."
Call    : generate_with_retry(client, formatted_prompt)
Output  : Markdown-style roadmap text + model_used = Gemini label
Display : shown inside the response box of index.html
```

8. Outcome

All five tools are independent, validated and exposed through one endpoint. Their text output, together with the model label, is what the Frontend Development stage presents to the student.

Frontend Development

EduGenie - AI Learning Assistant

1. Objective

The frontend is the part of EduGenie that students actually see. It is a single, server-rendered page built with HTML, CSS and Jinja2. The goal was a clean interface where a learner can pick a tool, type a query and read the result without any training.

2. Technologies Used

Technology	File	Role
HTML5	templates/index.html	Page structure, form and answer area
CSS3	static/style.css	Layout, colours, spacing, hover effect
Jinja2	templates/index.html	Fills in the response, prompt, task and model badge; keeps the selected option
FastAPI static mount	Main.py	Serves /static/style.css to the browser
Jinja2Templates	Main.py	Renders index.html for GET / and POST /generate

3. Page Structure

index.html is a standard HTML5 page with a UTF-8 character set and a responsive viewport meta tag. Everything is placed inside one div with the class **container**:

Element	Content in the code	Purpose
Heading (h1)	"EduGenie" with a genie icon	Brand name
Subtitle (p.subtitle)	"Multi-Model AI Learning Portal"	Tells the user the portal uses more than one model
Task selector (select#task)	Five options: Concept Explanation (Local LaMini-Flan-T5), General Q&A, Summarization, Quiz Generation, Learning Path Generation (each Gemini 3.8 flash)	Chooses the tool and shows which model will answer
Query box (textarea#prompt)	Placeholder "Type a concept, question, or topic...", required	Input for the question, topic or text
Submit button	"Submit Request"	Posts the form to /generate
Answer box (div.response-box)	Model badge, "Output:" label and the response	Shows the result; displayed only when a response exists

4. How the Template Connects to the Backend

Template code	Backend value	Effect
{% if task == 'qa' %}selected{% endif %} on each option	task	The drop-down keeps the task the user just used.
{{ prompt }} inside the textarea	prompt	The query stays in the box, so the user can edit and resubmit.
{% if response %} ... {% endif %}	response	The answer box is hidden on first visit and shown after a request.
Model: {{ model_used }}	model_used	Shows LaMini, the Gemini label, or System Error.
{{ response }}	response	Prints the answer. Jinja2 escapes HTML by default, so generated text cannot inject markup into the page.

The form uses method="post" and action="/generate", and its field names task and prompt must match the parameters in Main.py. The option values (concept, qa, summary, quiz, path) must match the TaskType enum exactly.

5. Styling (style.css)

Rule	Setting	Why it helps
body	Arial, background #f4f6f9, 40px padding, flex centering	Soft grey page with the card centred horizontally
.container	White, max-width 750px, 30px padding, 10px radius, light shadow	A readable card width that looks like an app panel
h1 and .subtitle	Dark blue-grey title, grey subtitle, centred	Clear hierarchy at the top
label	Bold, block, dark blue-grey	Labels sit above the fields and are easy to read
select, textarea	Full width, 10px padding, 6px radius, border-box	Consistent, comfortable inputs
textarea	110px height, vertical resize only	Enough room for typing without breaking the layout
button	Full width, blue #3498db, white bold text; darker #2980b9 on hover	A clear primary action with visual feedback
.response-box	Light blue background, 5px blue left border, pre-wrap, line-height 1.6	Stands out as the result; keeps the line breaks and bullet layout of the AI answer
.badge	Grey rounded label, small bold text	Shows the model name without distracting from the answer

6. User Interaction Flow

Landing	GET / shows the form with Concept selected and no answer box.
Select and type	The user chooses a task and enters a query.

Submit	A normal form POST is sent to /generate; the page reloads when the server replies.
Waiting	The first Concept request is slower because the local model loads; there is no loading indicator yet.
Result	The same page returns with the answer, the model badge, the selected task and the original query.

7. Why this Frontend Design is Useful

- **Low learning curve:** one form, two inputs and one button.
- **Transparency:** the drop-down and the badge both name the model used.
- **Continuity:** the selected task and the query are kept after submitting.
- **Reliable display of structured answers:** pre-wrap keeps quizzes, answer keys and roadmaps readable.
- **Lightweight:** no JavaScript framework, so the page loads quickly and is easy to maintain.

8. Current Limitations and Possible Improvements

- Gemini answers use Markdown symbols (headers, bullets). The page prints them as plain text, so a Markdown renderer would make results cleaner.
- The CSS has no media queries; the card adapts through width: 100% and max-width, but a dedicated mobile layout could be added.
- A loading spinner or disabled button would show that a request is in progress.
- A copy button and a history of earlier queries would improve everyday use.

Testing and Deployment

EduGenie - AI Learning Assistant

1. Introduction

This document explains how EduGenie is tested and how it is deployed. EduGenie depends on a local model, a cloud API, a web server and a browser page, so testing checks each layer separately and then the whole flow. Deployment makes sure the API key, the heavy PyTorch dependency and the first-run model download are handled correctly.

2. Testing

2.1 Testing Approach

Level	What is tested	Why it is useful
Unit	Each module alone (explanation_module, qna, summary_module, quiz_module, learning_path), including empty input	Finds mistakes in one feature without involving the web layer
Integration	POST /generate routes each task to the right module and shows the right model label	Confirms the pieces are connected correctly
Functional (UI)	The page in a browser: drop-down, textarea, submit button, answer box, badge	Confirms the student experience
Error and robustness	Missing or wrong key, API failure, invalid task, empty input	Confirms the page still renders with a clear message

2.2 Test Environment

Item	Details
Framework	Python 3, FastAPI, Jinja2
Server	Uvicorn
Local model	MBZUAI/LaMini-Flan-T5-783M (Transformers + PyTorch, CPU)
Cloud model	Gemini Flash through google-genai; key in GEMINI_API_KEY
Browser	Any modern browser such as Chrome, Edge or Firefox

2.3 Test Cases

ID	Test case	Input	Expected result
TC-01	Home page	GET /	index.html loads; Concept selected; no answer box
TC-02	Concept	task=concept, "Photosynthesis"	Short explanation; badge shows LaMini-Flan-T5-783M (Local CPU)

ID	Test case	Input	Expected result
TC-03	Q&A	task=qa, "What is recursion?"	Detailed answer with examples; Gemini badge
TC-04	Summary	task=summary, a paragraph	1-2 sentence overview and bullet takeaways
TC-05	Quiz	task=quiz, "Python lists"	3 questions with options A-D and an Answer Key
TC-06	Learning path	task=path, "Machine Learning"	Phased roadmap with topics, projects and timelines
TC-07	Empty concept input	task=concept, spaces only	"Please provide a concept or topic to explain."
TC-08	Empty Gemini input	qa / summary / quiz / path, spaces only	Task-specific "Please provide ..." message; no API call
TC-09	Missing field	POST without prompt	422 validation error
TC-10	Invalid task	task="unknown"	422 error from the TaskType enum
TC-11	No API key	Start without GEMINI_API_KEY	RuntimeError at start-up
TC-12	OAuth token as key	Key starting with ya29.	RuntimeError asking for an AI Studio key
TC-13	API failure	Invalid key or no network	"Error generating response: ..." and badge System Error
TC-14	First model load	First concept request	Model loads once; later requests are faster
TC-15	Long input	Concept prompt over 512 tokens	Input truncated to 512 tokens; output at most 256 new tokens
TC-16	Form state kept	Submit any request	Selected task and query are still shown after the page reloads
TC-17	Output safety	Prompt that makes the model output HTML tags	Tags are shown as text because Jinja2 escapes the response

2.4 Automated Route Tests

Route tests can be automated with pytest and FastAPI's TestClient. The Gemini function is replaced with a mock, so tests run without using the API quota.

```
pip install pytest httpx

# test_app.py
from fastapi.testclient import TestClient
from unittest.mock import patch
import Main

client = TestClient(Main.app)

def test_home():
    assert client.get('/').status_code == 200

def test_invalid_task():
    r = client.post('/generate', data={'prompt': 'x', 'task': 'bad'})
    assert r.status_code == 422

def test_qa_routing():
```

```
    with patch('Main.qna.get_answer', return_value='mock answer'):
        r = client.post('/generate', data={'prompt': 'hi', 'task': 'qa'})
    assert 'mock answer' in r.text

# run:  pytest -v
```

Importing Main requires a GEMINI_API_KEY value (it can be a dummy string for route tests) and existing templates and static folders, because Main.py checks the key and mounts those folders when it is imported.

2.5 Observations and Limitations

- The local model gives short answers, so its quality is lower than the Gemini tasks.
- Gemini output varies between runs, so tests should check structure (for example the presence of an Answer Key) rather than exact wording.
- The first concept request is slow because the model loads into memory.
- All four cloud tasks depend on network access and API quota.

3. Deployment

3.1 Requirements

Requirement	Description
Python	3.9 or higher
Packages	Everything in requirements.txt
API key	Gemini key from Google AI Studio in the .env file or an environment variable
Memory	Around 4 GB free is recommended for PyTorch and the local model (estimate)
Network	Internet for Gemini calls and the one-time Hugging Face model download

3.2 Project Structure

```
EduGenie/
|-- Main.py
|-- explanation_module.py
|-- qna.py
|-- summary_module.py
|-- quiz_module.py
|-- learning_path.py
|-- gemini_helper.py
|-- templates/index.html
|-- static/style.css
|-- requirements.txt
|-- .env                (never commit this file)
```

3.3 Local Deployment

- Create and activate a virtual environment.
- Run pip install -r requirements.txt.

- Create .env containing GEMINI_API_KEY=your_api_key_here.
- Start the server: uvicorn Main:app --host 127.0.0.1 --port 8000 --reload
- Open http://127.0.0.1:8000 in the browser.

The file is named Main.py with a capital M, so the Uvicorn target must be written Main:app. On case-sensitive systems such as Linux, main:app would not be found.

3.4 Production Deployment

- Run without --reload, for example: uvicorn Main:app --host 0.0.0.0 --port 8000 --workers 2. Each worker loads its own copy of the local model, so memory grows with the worker count.
- Put Nginx or another reverse proxy in front for HTTPS and static files.
- Provide the API key as an environment variable or platform secret instead of a file in the repository.
- Pre-download the model during the build step, or keep a persistent cache, so the first user does not wait.
- Run the process under a supervisor such as systemd so it restarts after a crash.

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "Main:app", "--host", "0.0.0.0", "--port", "8000"]

# docker build -t edugenie .
# docker run -p 8000:8000 -e GEMINI_API_KEY=your_key edugenie
```

3.5 Post-Deployment Checklist

- The home page opens and style.css loads (the card is centred and styled).
- One request for each of the five tasks works and shows the correct badge.
- Logs show the local model loading once without errors.
- The API key does not appear in the repository, logs or page source.
- HTTPS is active and the server restarts automatically after a failure.

Conclusion

EduGenie - AI Learning Assistant

1. Summary of the Project

EduGenie is a multi-model AI learning portal built with FastAPI, Jinja2, HTML and CSS. A student opens one page, chooses a task, enters a question, topic or text, and receives a result together with the name of the model that produced it. The portal offers five study aids: concept explanation, question answering, summarisation, quiz generation and learning-path creation.

The project is built on a hybrid model strategy. The simplest task, concept explanation, runs on the local LaMini-Flan-T5-783M model, so it has no per-request cost. The four tasks that need longer or more strictly structured output run on Gemini through the google-genai client.

2. How the System Was Built

Stage	What was done	Benefit
Model Selection and Architecture	Chose the local + cloud model mix and a layered, one-module-per-task structure	Low cost, clear code, easy to extend
Core Functionalities Development	Built five task modules with prompts, validation, a retry helper, routes and error handling	Each learning tool works independently and fails safely
Frontend Development	Built index.html and style.css with a task selector, query box and result box with model badge	Simple page that anyone can use
Testing and Deployment	Defined test cases, route tests and local/production deployment steps	Repeatable checking and set-up

3. Objectives Achieved

Objective	Outcome
Explain concepts quickly	Local model gives a short explanation with no API cost.
Answer questions in depth	qna.py requests detailed answers with practical examples.
Save reading time	summary_module.py produces an overview and bullet takeaways.
Test understanding	quiz_module.py creates a 3-question quiz with an Answer Key.
Plan long-term study	learning_path.py produces a phased roadmap with projects and timelines.
Stay reliable	Start-up key checks, empty-input checks, retry helper and a route-level error handler keep the page working.
Be transparent	The drop-down and the answer badge always name the model.

4. Key Strengths

- **Modular design:** adding or changing a feature touches one module.
- **Hybrid models:** cost, speed and quality are balanced by using each model where it fits best.
- **Safe configuration:** the key comes from the environment and is validated at start-up.
- **Efficient resource use:** the local model is loaded only when first needed and then reused.
- **Readable results:** the response box keeps the line breaks of quizzes and roadmaps.
- **Secure output:** Jinja2 escapes generated text before it is placed in the page.

5. Limitations

- The local model produces short answers and is less accurate on complex topics.
- Four of five features need internet access and Gemini quota.
- Markdown from Gemini is displayed as plain text, not rendered.
- There are no user accounts, so history and scores are not saved.
- The quiz is fixed at three questions and cannot be scored in the page.
- There is no loading indicator, and the first concept request is slow while the model loads.
- AI-generated content may contain mistakes and should be checked against reliable sources.

6. Future Enhancements

- Render Markdown in the response box and add a copy button.
- Add a loading indicator and stream answers as they are generated.
- Make quizzes interactive with scoring, adjustable length and difficulty.
- Add user accounts with history, bookmarks and progress tracking.
- Accept uploaded PDF or Word notes for summaries and quizzes.
- Add multilingual support and text-to-speech for accessibility.
- Cache repeated requests and add rate limiting to control cost.
- Offer GPU support or a larger local model for better offline explanations.

7. Final Remarks

EduGenie shows how language models can become a practical study companion with a compact, well-organised codebase. By combining a free local model with a stronger cloud model, it gives learners explanations, answers, summaries, quizzes and study roadmaps in one place. With personalisation, progress tracking and richer interaction added, it can grow from a working prototype into a complete AI-powered learning platform.